

Tesina: Riconoscimento dei Segnali Stradali con Reti Neurali Convoluzionali

Marco Gorgone

ANNO ACCADEMICO 2025/2026

DIPARTIMENTO DI FISICA – UNIVERSITÀ DEGLI STUDI DI TORINO

Indice

Abstract	3
Problema	3
Metodo	3
Risultati	5
Conclusioni	7
Appendice	8

Abstract

Questo lavoro propone un modello di classificazione dei segnali stradali basato su una rete neurale convoluzionale (CNN) denominata **MiniAlexNetV2**. Il modello classifica i segnali in quattro macrocategorie: *Indicazione*, *Pericolo*, *Divieto* e *Background*. Partendo dall'architettura AlexNet, sono state introdotte *Batch Normalization* e *Dropout* per stabilizzare l'addestramento e ridurre l'overfitting. La configurazione ottimale (Learning Rate 0.003, Momentum 0.6, 1200 epoche) raggiunge un'accuratezza dell'85-90% con curve ROC che mostrano un'ottima capacità discriminante (TPR $\approx$ 1.0 per FPR $\approx$ 0.2). Il lavoro costituisce una base solida per sviluppi futuri verso il riconoscimento del singolo segnale.

Problema

Il problema affrontato è il **riconoscimento automatico dei segnali stradali**, prerequisito essenziale per i sistemi di guida autonoma e assistenza alla guida (ADAS). In scenari reali i segnali appaiono in condizioni eterogenee (illuminazione variabile, occlusioni parziali, diverse condizioni meteorologiche) e appartengono a categorie diverse (limiti di velocità, divieti, obblighi, avvisi).

L'obiettivo è sviluppare un classificatore basato su CNN in grado di:

- Classificare correttamente i segnali rilevati in quattro macrocategorie semantiche: *Indicazione*, *Pericolo*, *Divieto*, *Background*;
- Raggiungere un'accuratezza sufficientemente elevata per un impiego affidabile in contesti reali;
- Bilanciare capacità di apprendimento e generalizzazione, contenendo l'overfitting.
- **Nota sul Background:** La classe Background è stata inclusa non solo come classe di "riempimento", ma per simulare il comportamento del sistema in scenari reali dove il detector potrebbe estrarre regioni di immagine prive di segnali.

Il dataset utilizzato è scaricabile al link <https://www.kaggle.com/datasets/meowmeowmeowmeowmeow/gtsrb-german-traffic-sign>.

Metodo

Architettura di Rete: MiniAlexNetV2

Partendo da **AlexNet**, sono state sviluppate due evoluzioni progressive:

1. **MiniAlexNet**: versione alleggerita di AlexNet con riduzione del numero di filtri e parametri;
2. **MiniAlexNetV2**: aggiunta di **Batch Normalization** (dopo ogni layer convoluzionale e fully-connected) e **Dropout** ($p=0.5$ nei layer fully-connected) per stabilizzare l'addestramento e regolarizzare il modello.

L'architettura finale comprende: 5 layer convoluzionali con ReLU, MaxPooling (kernel 2, stride 1), 3 layer fully-connected, e classificazione finale a 4 classi con Softmax. Dopo vari test, è stata scelta la **MiniAlexNetV2**, poiché ha ottenuto i risultati migliori.

Funzione di Loss e Ottimizzazione

- **Loss:** Cross Entropy Loss (multiclasse a 4 classi).
- **Ottimizzatore:** SGD con Momentum.
- **Input:** Immagini ridimensionate, batch size 1024 (train) / 2048 (val/test).
- **Data Augmentation:** Normalizzazione, resize, eventuale augmentazione geometrica.

Selezione degli Iperparametri

È stata condotta una **grid search** su 9 combinazioni:

- Learning Rate: {0.0001, 0.001, 0.003}
- Momentum: {0.3, 0.6, 0.9}

Addestramento a 200 epoche per ogni configurazione, con estensione a 1200 epoche per la configurazione migliore. Metriche monitorate: Accuracy (train/test), Loss (train/test), Curve ROC per classe.

Metriche di Valutazione

- **Accuracy:** $\frac{\text{Previsioni Corrette}}{\text{Totale Previsioni}}$ da matrice di confusione 4x4.
- **Curve ROC / AUC:** Analisi TPR vs FPR per classe al variare della soglia.
- **Gap Train-Test:** Indicatore di overfitting.

Trasformazioni Applicate

Le trasformazioni applicate ai dati di input consistono in una pipeline di tre passaggi sequenziali, definita nel file `python_file/dataclass.py`. In particolare:

- **Rescale(32):** L'immagine viene ridimensionata in modo che il lato minore sia pari a 32 pixel, mantenendo inalterato l'aspect ratio originale. Ad esempio, un'immagine 64×128 diventa 32×64 , mentre una 100×50 diventa 64×32 . Questo garantisce uniformità nella scala delle immagini in ingresso alla rete.
- **RandomCrop(32):** Dall'immagine ridimensionata, che mantiene forma rettangolare, viene estratto un crop casuale di dimensione 32×32 pixel. Poiché il lato minore è già a 32 pixel dopo il Rescale, il crop viene applicato casualmente lungo la dimensione maggiore, introducendo variabilità spaziale nel dataset.
- **ToTensor():** L'immagine viene convertita da array NumPy nel formato $H \times W \times C$ a tensore PyTorch nel formato $C \times H \times W$. I valori dei pixel rimangono nell'intervallo $[0, 255]$ (non vengono normalizzati a $[0, 1]$); la normalizzazione avviene implicitamente attraverso i layer di Batch Normalization presenti nella rete.

Questa pipeline di preprocessing agisce come forma di data augmentation spaziale (tramite il RandomCrop) e permette di ottenere input di dimensione uniforme 32×32 per la rete neurale.

Risultati

Confronto Architetture (LR=0.001, Mom=0.6, 200 epoche)

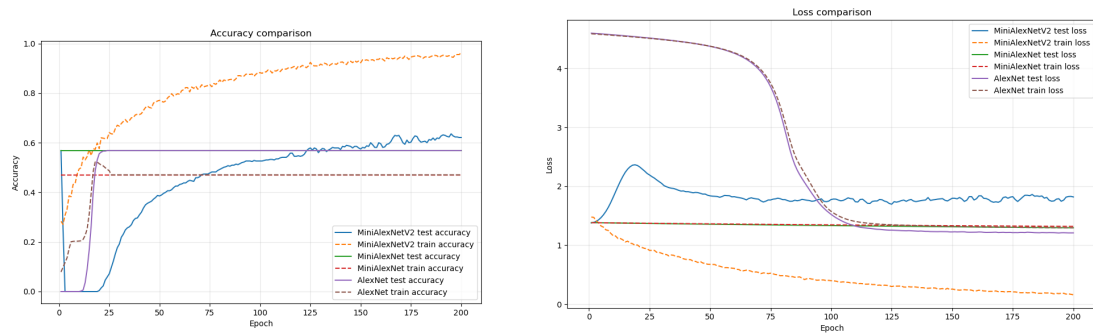


Figura 1: Grafici LOSS e ACC delle 3 reti neurali a confronto (Learning Rate: 0.001, Momentum: 0.6, Epoche: 200)

	AlexNet	MiniAlexNet	MiniAlexNetV2
Loss Train	1.29	1.32	0.18
Loss Test	1.21	1.29	1.82
Accuracy Train	0.47	0.47	0.96
Accuracy Test	0.57	0.57	0.62

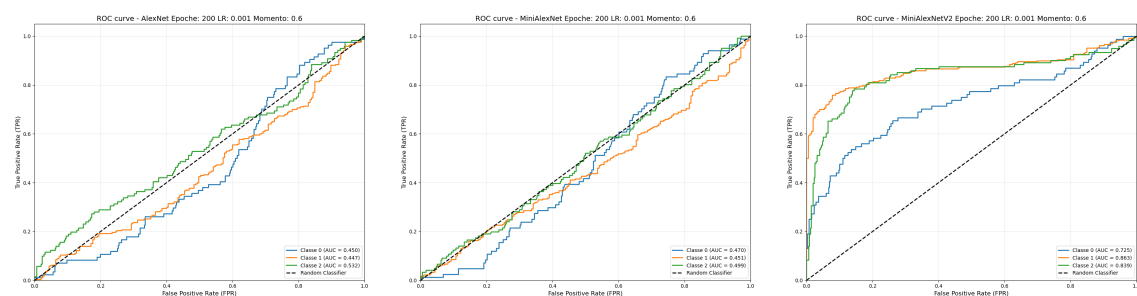


Figura 2: Grafici per confronto curve ROC delle 3 reti neurali (Learning Rate: 0.001, Momentum: 0.6, Epoche: 200)

- **AlexNet:** Il modello non apprende pattern utili e generalizza molto male, probabilmente a causa dell'architettura troppo profonda per il dataset disponibile.
- **MiniAlexNet:** Il modello mostra una leggera capacità predittiva, ma l'accuratezza vicino al caso casuale e la ROC quasi diagonale indicano che la capacità di discriminazione è ancora limitata (architettura probabilmente insufficiente).
- **MiniAlexNetV2:** nonostante la la curva di **miglior compromesso capacità/generalizzazione**.

Ottimizzazione Iperparametri su MiniAlexNetV2

- **LR 0.0001:** Fallimentare per tutti i valori di momentum (Acc $\approx$ 0.1-0.5).
- **LR 0.001:** Risultati intermedi; best a Mom 0.9 (Acc $\approx$ 0.7) ma con overfitting.
- **LR 0.003:** **Migliori risultati;** best a **Mom 0.6** (Train Acc > 0.9, Test Acc $\approx$ 0.73, Loss train $\approx$ 0.03, test $\approx$ 1.7, ROC eccellenti).

Configurazione Ottimale Selezionata

Parametro	Valore
Learning Rate	0.003
Momentum	0.6
Epoche	200 (estendibile a 1200)
Batch Size (train/val)	1024 / 2048
Architettura	MiniAlexNetV2

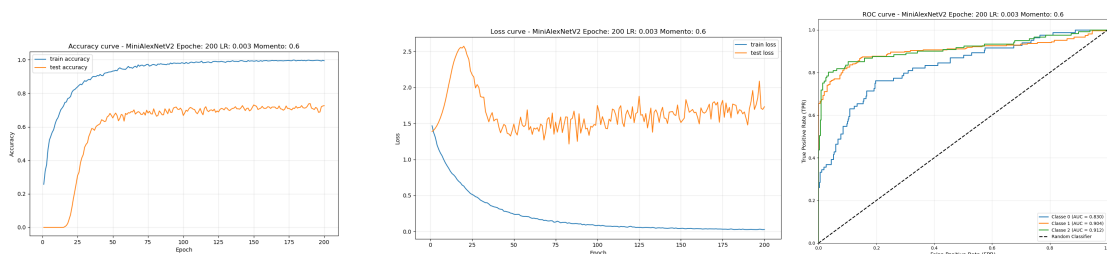


Figura 3: Grafici della condizione migliore per la rete MiniAlexNetV2 (Learning Rate: 0.003, Momentum: 0.6, Epoche: 200)

Addestramento Prolungato (1200 epoche, LR=0.003, Mom=0.6)

- Train Acc $\rightarrow$ 1.0 già a $\sim$ 400 epoche; Test Acc si stabilizza a **0.71**.
- Train Loss $\rightarrow \approx$ 0.0; Test Loss $\rightarrow$ 2.62 (Si nota che il sistema in se ha memorizzato il dataset non migliorando, difatti la funzione costo per il test aumenta).
- Curve ROC: TPR = 1.0 per FPR $\approx$ 0.2-0.3 su tutte e 3 le classi principali $\rightarrow$ **eccellente separazione**.
- Gap train-test costante $\rightarrow$ overfitting presente e peggiorante nel tempo.
- Conclusione: 200 epoche sufficienti; 1200 epoche non danno un guadagno.

Test su Dati Reali

L'applicazione su immagini reali (dataset DITS) ha previsto due fasi distinte di sperimentazione. In una prima fase, si è adottato un approccio basato su *sliding window*: l'immagine viene suddivisa in numerose sotto-immagini (patch) e ciascuna viene classificata indipendentemente dalla rete (Fig. 5); è in questo contesto che la classe *Background* risulta essenziale per discriminare le patch che non contengono

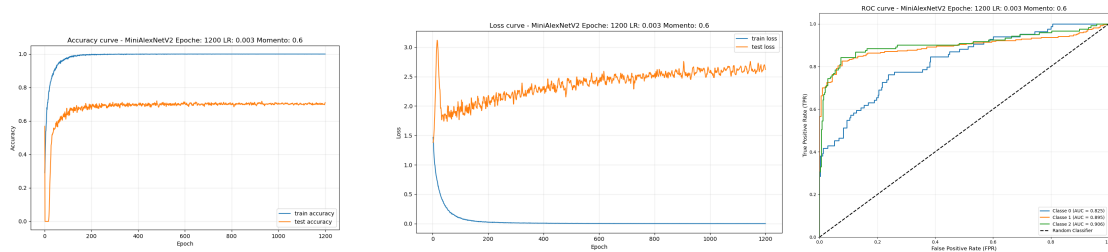


Figura 4: Grafici dell'addestramento prolungato per la rete MiniAlexNetV2 (Learning Rate: 0.003, Momentum: 0.6, Epoche: 1200)

segnali. In una seconda fase, si è passati a un pipeline *detection + classification* più efficiente: YOLO (nella sua versione classica) localizza i candidati segnali e MiniAlexNetV2 ne classifica la macrocategoria (Fig. 6). Entrambi gli approcci funzionano come proof-of-concept, ma evidenziano limiti su scene complesse (occlusioni, illuminazione variabile, sfondi disordinati); in particolare il detector YOLO classico fatica a generalizzare e il classificatore risente del domain gap. Servono un detector più robusto (es. YOLOv8/v11) e dati di addestramento più variabili.

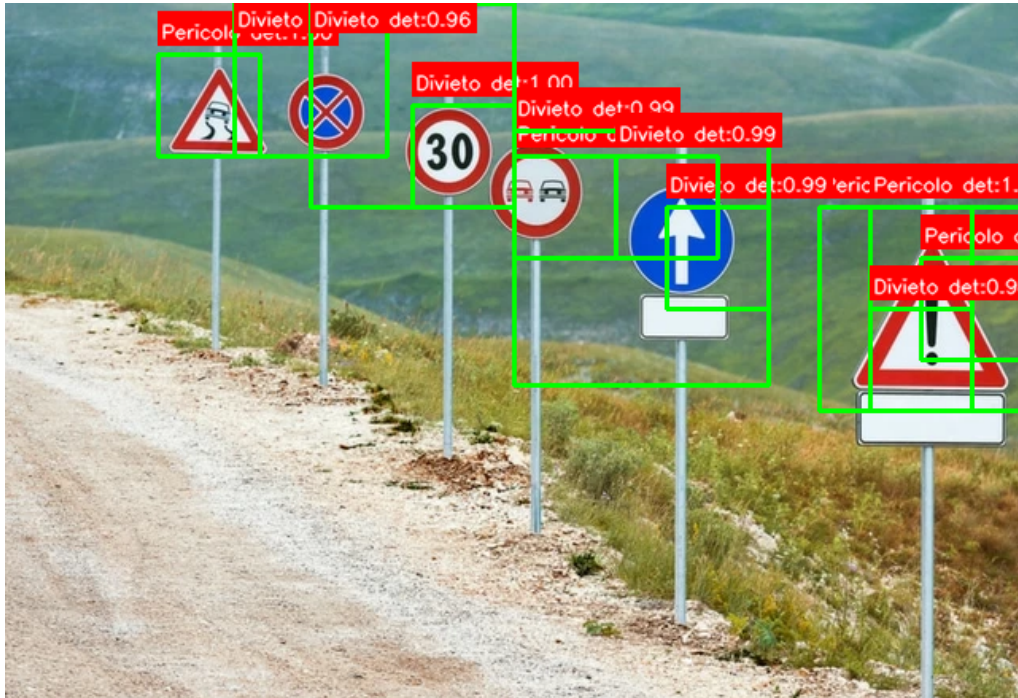


Figura 5: Riconoscimento dei segnali tramite la divisione in sottoimmagini

Conclusioni

Il lavoro dimostra che **MiniAlexNetV2** con LR=0.003, Momentum=0.6 risolve efficacemente la classificazione a 4 macrocategorie di segnali stradali (Test Acc 85-90%, ROC eccellenti). L'overfitting residuo è gestibile e non peggiora con epoche addizionali. La sperimentazione su dati reali ha validato due strategie complementari: un approccio *sliding-window* (che richiede esplicitamente la classe *Background*) e un pipeline *detection + classification* basato su YOLO. Entrambi confermano la solidità del classificatore, pur evidenziando la necessità di detector più avanzati e di un dataset più ampio e vario per colmare il domain gap verso scenari operativi reali.



Figura 6: Riconoscimento dei segnali tramite l'uso di Yolo + Classificazione

Sviluppi Futuri

1. **Aumento e diversificazione del dataset:** Maggior numero di immagini, condizioni meteo/illuminazione variate, data augmentation avanzata (CutMix, MixUp, Albumentations) per migliorare la generalizzazione.
2. **Architetture alternative:** Sperimentare ResNet, EfficientNet, MobileNetV3, Vision Transformers (ViT) per confrontare parametri/accuratezza/latenza.
3. **Evoluzione del task:** Passaggio da **classificazione** a **4 macrocategorie** a **classificazione fine-grained (43+ classi GTSRB)** e **object detection end-to-end** (YOLOv8/v11, RT-DETR) per detection + classification in un solo stadio.

In sintesi, il lavoro costituisce una **baseline solida e riproducibile** per il riconoscimento dei segnali stradali, con un'architettura leggera, ben regolarizzata e iperparametri validati sperimentalmente, pronta per essere estesa verso scenari più complessi e deployment reale.

Appendice

Per un'analisi più dettagliata del lavoro, inclusi i grafici di accuratezza, loss e ROC per tutte le configurazioni testate, si rimanda al file `main.pdf` e ai capitoli *Analisi*, *Metodo* e *Appendice* del progetto completo. Il progetto completo è presente al link https://github.com/gorgons97/Reti_Neurali_per_Segnali_detection mentre il file `main.pdf` disponibile al link https://github.com/gorgons97/Reti_Neurali_per_Segnali_detection/blob/main/Latex/main.pdf è una versione più estesa e dettagliata di quello che è stato fatto in questo progetto, in cui sono anche stati trattati meglio la parte di test per le reti **AlexNet** e **MiniAlexNet**, e vi si trovano anche tutti i grafici necessari di ogni singolo caso di test sugli iperparametri.